INDEX

S. NO.	NAME OF EXPERIMENT	DATE	PAGE NO.	REMARKS
1.	Practice Bash, I/O Redirection, pipe, File and directory operations, searching, process management & Text editing	25/08/26	1-4	Done
2.	Use the data, who commands in one line such that the output data is displayed on the screen.	03/09/26	5-6	Done
3.	To write a shell script that takes a command-line argument & reports the owner of the given argument	08/09/26	7-9	Done
4.	write a shell script that determines the period for which a specified user is working on the system	15/09/26	9-10	Done
5	write a shell script and C-program to perform string operations	22/09/26	11	
6	write a C program that takes as a command line-argument	29/09/26	12	
7	write a CPU bound C-program and a I/O bound C program	22/10/26	13	
8	observe and understand the process map of the previous	22/10/26	14	
9	write a program in C that creates a child and list in PID	27/10/26	15	
10	observing virtual memory address across concurrent.	29/10/26	16	

```
@harnoorkour:~$ pwd
oxuser
@harnoorkour:~$ ls
 Documents  Downloads  Music  Pictures  Public  Templates  Videos  file.txt  snap
@harnoorkour:~$ man ls
@harnoorkour:~$ mkdir MIET
@harnoorkour:~$ cd MIET
@harnoorkour:~/MIET$ mkdir CSE AIML CYBER
@harnoorkour:~/MIET$ ls
SE  CYBER
@harnoorkour:~/MIET$ cd ..
@harnoorkour:~$ touch file1.txt
@harnoorkour:~$ echo "HELLO"

@harnoorkour:~$ cat > file1.txt
veryone
@harnoorkour:~$ cat >> file1.txt
earning OS
@harnoorkour:~$ cat file1.txt
veryone
earning OS
@harnoorkour:~$ grep "are" file1.txt
earning OS
@harnoorkour:~$ ls | grep "file1"
xt
@harnoorkour:~$ cp file1.txt file.txt
@harnoorkour:~$ cat file.txt
veryone
earning OS
@harnoorkour:~$ mv file.txt file2.txt
@harnoorkour:~$ cat file2.txt
veryone
earning OS
@harnoorkour:~$ cat file1.txt file2.txt > combined.txt
@harnoorkour:~$ sort combined.txt
veryone
veryone
earning OS
earning OS
@harnoorkour:~$ head -n 1 file2.txt
veryone
@harnoorkour:~$ cat < combined.txt
eryone
earning OS
eryone
```

civil
cse
aiml
com-301
sessional-1
sessional-2
com-302
com-303
com-304
cyber
management

```
harnoorkour:~$ ccd /root &> files.txt
harnoorkour:~$ ls
Documents  Downloads  MIET  Music  Pictures  Public  Templates  Videos  combined.txt  file1.txt  file2.txt  file
harnoorkour:~$ rmdir MIET
lled to remove 'MIET': Directory not empty
harnoorkour:~$ rm -r MIET
harnoorkour:~$ ls
```

LABORATORY RECORD

DATE 25-08-26 EXP. No. 1

Title: Practice Basic Bash, I/O Redirection, Pipe, File and Directory Operations Searching files, Process Management and Text Editors.

Theory:

→ pwd: displays the current absolute path of working directory.

→ cd: changes the current directory to MIET

→ cd ..: moves up one directory level towards the root directory.

→ mkdir MIET: creates a new directory named MIET.

→ mkdir CSE AI/ML + cyber: Creates Multiple directories in current path.

→ rmdir MIET: removes an empty directory named MIET.

→ rmdir -r MIET: recursively deletes a directory alongside all of its nested contents.

→ ls: lists visible files and folders in current directory.

→ ls -l: displays detailed file information; ls -a: lists all file including hidden.

→ touch file1.txt: creates any empty file file1.txt

→ echo "word": prints the text in terminal output.

→ cp file1.txt file.txt: copies source file content into a new destination file.

→ mv file.txt file2.txt: renames a file; cat file1.txt: display entire content of a file.

→ head -n 1 file2.txt: displays only the top specified no. of lines of a file.

→ sort file.txt: reads a file and prints in alphabetic order.

→ cat > file1.txt: overwrites a file; cat >> file1.txt: appends new lines

→ ls | grep "file1": sends the output of ls as input to grep using a pipe (|) to filter for specific text.

→ tree: used to display folders structure of a directory as a visual tree-like diagram.

DATE 01-09-26

EXP. No. 1 (b)

Title : Practice Basic Bash, Process Management

Theory:

→ Jobs : checks the running process

→ Sleep Time : the terminal stops till given time.

→ Sleep Time & : tells the terminal to wait till the time given while running that task quietly in background.

→ Kill % 'No.' / Kill PID : Kills the process

→ fg PID / % 'No.' : for running the process in foreground.

→ bg PID / % 'No.' : for running the process in background

→ ps : gives the information and status of processes.

→ ps -e : gives the status of hidden processes.

→ ps -ef : gives also the parent process ID.

→ ps aux : Shows the memory allocation of the process but not real time.

→ Kill -9 PID : Kills the process immediately

→ Kill -15 PID : Kills the process after saving

→ ps -o pid, ni, cmd : display a customized list of currently running terminated process, showing their pid, nice value (priority). and command name.

DATE

EXP. No.

```
vboxuser@harnoorkour:~$ ps
    PID TTY          TIME CMD
  36484 pts/0    00:00:00 bash
  77631 pts/0    00:00:00 sleep
  79482 pts/0    00:00:00 sleep
  79484 pts/0    00:00:00 sleep
  79512 pts/0    00:00:00 ps
vboxuser@harnoorkour:~$ ps -e
    PID TTY          TIME CMD
      1 ?        00:00:21 systemd
      2 ?        00:00:00 kthreadd
      3 ?        00:00:00 pool_workqueue_release
      4 ?        00:00:00 kworker/R-rcu_gp
      5 ?        00:00:00 kworker/R-sync_wq
      6 ?        00:00:00 kworker/R-kvfree_rcu_reclaim
      7 ?        00:00:00 kworker/R-slub_flushwq
      8 ?        00:00:00 kworker/R-netns
     13 ?        00:00:00 kworker/R-mm_percpu_wq
     14 ?        00:00:05 ksoftirqd/0
     15 ?        00:00:02 rcu_preempt
     16 ?        00:00:00 rcu_exp_par_gp_kthread_worker/0
```

```
vboxuser@harnoorkour:~$ sleep 500 &
[3] 79482
vboxuser@harnoorkour:~$ sleep 400 &
[4] 79484
vboxuser@harnoorkour:~$ jobs
[2]+  Stopped                 sleep 200
[3]-  Running                 sleep 500 &
[4]+  Running                 sleep 400 &
vboxuser@harnoorkour:~$ sleep 600 &
[5] 79494
vboxuser@harnoorkour:~$ kill %4
[4]   Terminated              sleep 400
vboxuser@harnoorkour:~$ bg %5
bash: bg: job 5 already in background
[5]+ Terminated
vboxuser@harnoorkour:~$
```


```
vboxuser@harnoorkour:~$ sleep 500 &
[3] 79482
vboxuser@harnoorkour:~$ sleep 400 &
[4] 79484
vboxuser@harnoorkour:~$ jobs
[2]+  Stopped                 sleep 200
[3]-  Running                 sleep 500 &
[4]+  Running                 sleep 400 &
vboxuser@harnoorkour:~$ sleep 600 &
[5] 79494
vboxuser@harnoorkour:~$ kill %4
[4]   Terminated              sleep 400
vboxuser@harnoorkour:~$ bg %5
bash: bg: job 5 already in background
```


```
vboxuser@harnoorkour:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1       0  0 09:18 ?        00:00:21 /usr/lib/systemd/systemd --system --deserialize=92
root           2       0  0 09:18 ?        00:00:00 [kthreadd]
root           3       2  0 09:18 ?        00:00:00 [pool_workqueue_release]
root           4       2  0 09:18 ?        00:00:00 [kworker/R-rcu_gp]
root           5       2  0 09:18 ?        00:00:00 [kworker/R-sync_wq]
root           6       2  0 09:18 ?        00:00:00 [kworker/R-kvfree_rcu_reclaim]
root           7       2  0 09:18 ?        00:00:00 [kworker/R-slub_flushwq]
root           8       2  0 09:18 ?        00:00:00 [kworker/R-netns]
root          13       2  0 09:18 ?        00:00:00 [kworker/R-mm_percpu_wq]
root          14       2  0 09:18 ?        00:00:05 [ksoftirqd/0]
root          15       2  0 09:18 ?        00:00:02 [rcu_preempt]
root          16       2  0 09:18 ?        00:00:00 [rcu_exp_par_gp_kthread_worker/0]
```

DATE 03-09-2026 EXP. No. 02

Title: Use the date and who commands, in one line such that the output of data is displayed on the screen and the output of who is redirected do a file. With sed commands that swap the first and second word in each line in a file.

Theory:

→ **date command:** used to display the current system date & time.

→ **w/who command:** Used to display information about the current users logged into the system.

→ **sed command:** stands for stream editor. Used to search, modify, substitute, delete, insert & manipulate text in a file.

general syntax: sed 'command" filename

→ **Important sed command options:**

★ '-n': supresses automatic printing

★ 'p': use to explicitly printing a line

★ 'd': used to delete lines from the output

★ 's': substitute command is used to replace one piece of text with another.

★ '-i': used to make changes in the original file, not only the terminal.

★ '-ibak': makes changes in the original file & creates a similar backup file.

★ '[ ]': used to define character class. eg [abc]

★ '[^]': negated character class.

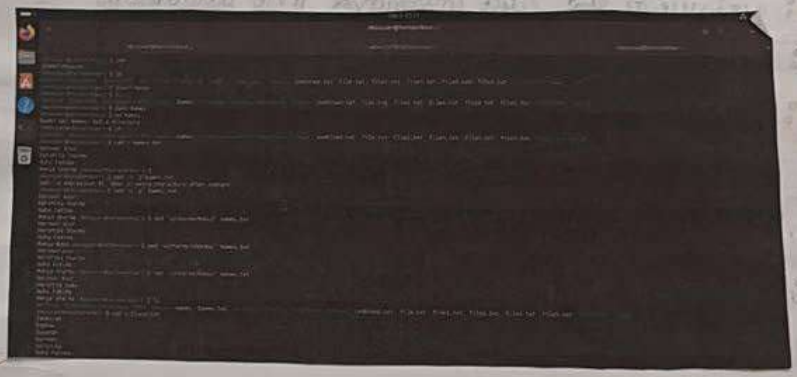
★ '[^]*': used to capture the first words

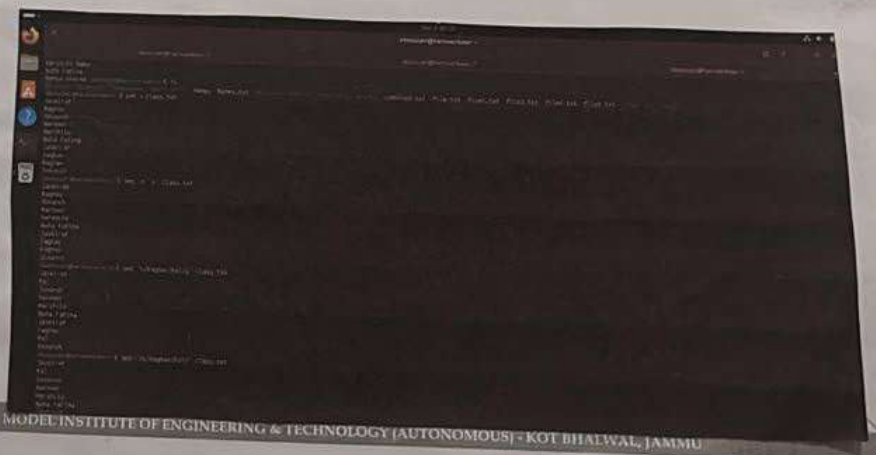
DATE..... EXP. No.....

general syntax: sed 's/[^]*/xyz' Names.txt

★ **Command to swap the first and second words**

general syntax: s/\([^]*\)/\([^]*\)/\2 \1/ Names.txt





DATE: 08-09-26 EXP. No. 03-a

Title: To write a shell script that takes a command-line argument and reports whether the given argument is a directory or a file.

Theory:

#!/bin/bash: specifies that the script should be executed using bash shell

→ **$1:** represents the first command line argument

→ **-d:** Tests whether the given part is directory.

→ **-f:** Tests whether if it's a file.

→ **if:** execute a block of code if its true.

→ **echo:** displays the text/output on terminal.

→ **fi:** makes end of the if-elif-else statement.

```
vboxuser@harnoorkour:~$ nano f1.sh
vboxuser@harnoorkour:~$ chmod +x f1.sh
vboxuser@harnoorkour:~$ ./f1.sh
Enter a file or directory:
Names.txt
Names.txt is a file.
vboxuser@harnoorkour:~$ ./f1.sh
Enter a file or directory:
Pictures
Pictures is a directory.
```

```
 GNU nano 8.7.1
#!/bin/bash
echo "Enter a file or directory: "
read n
if [ -d "$n" ]
then
        echo "$n is a directory."
elif [ -f "$n" ]
then
        echo "$n is a file."
else
        echo "$n is neither a file nor a directory."
fi
```

DATE: 10-09-26

EXP. No: 3-b

Title: Write a shell script that takes file names on arguments and convert all of them to Uppercase.

Theory:

i) **tr '[:lower:]' '[:upper:]'** : tr stands for transform. It is used to translate or replace characters in the input text. It is also used to convert uppercase letters to lower letters or vice-versa.

ii) **nano upper.sh** : It creates and open the shell script in the nano text editior for writing.

iii) **for** : The for loop is used to process each file name given as command-line argument one by one.

iv) **"$@"** : represents all the command-line arguments passed to shell script.

v) **mv** : move command is use to move or remove or to rename file or directories. Here rename file to uppercase.

vi) **done** : used to mark the end of for loop.

```
vboxuser@harnoorkour:~$ echo "hello world" | tr '[:lower:]' '[:upper:]'
HELLO WORLD
vboxuser@harnoorkour:~$ nano upper.sh
vboxuser@harnoorkour:~$ . upper.sh file3.txt file4.txt new.txt
file name  changed to upper case
file name  changed to upper case
file name  changed to upper case
vboxuser@harnoorkour:~$ ls
1.sh    Desktop     FILE1.TXT   FILE4.TXT   NEW.TXT     Templates   check.sh                    snap
2.sh    Documents   FILE2.TXT   MIET        Pictures    Videos      Comp.sh         new.sh      upper.sh
3.sh    Downloads   FILE3.TXT   Music       Public      cat         Continents.txt  num.sh      user.txt
vboxuser@harnoorkour:~$
```

```
 GNU nano 8.7.1                                  upper.sh *
#!/bin/bash
for file in "$@"
do
    if [ -f "$file" ]
    then
        mv "$file" "$(echo "$file" | tr '[:lower:]' '[:upper:]')"
        echo "file name changed to upper case"
    else
        echo "$file does not exist"
    fi
done
```

LABORATORY RECORD

DATE 15-9-25 EXP. No. 6a(4ca)

Title: Write a shell script that determines the period for which a specified user is waiting on the system.

Write a shell script that displays all the lines b/w start and end line numbers passed as argument.

Write a shell script that deletes all lines containing a specified word in one or more files supplied as arguments to it.

Theory:

who: displays a list of users currently logged it into the O.S.

awk: a text processing tool used to search, filter and print specific columns from command output

exit: Immediately terminates a shell script or exits a command loop once a match is found.

Sed: A stream editor used to process and filter text, such as printing specific line ranges from a file.

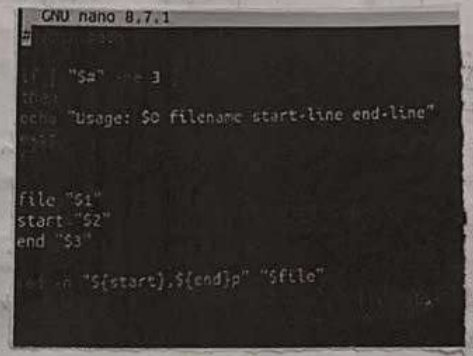
$#: A special shell variable that holds the total count of command-line argument passed to the script.

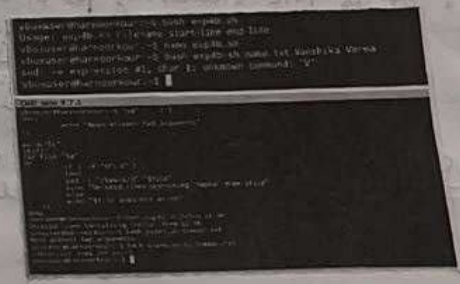
$1, $2, $3: Positional parameters that store the command-line augmented line like file name, start line, end line provided to the script.

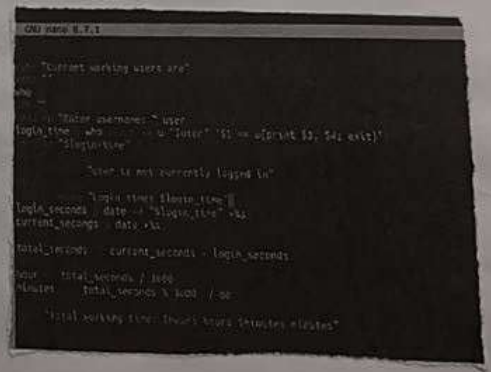
Shift: shifts command-line parameters to the left, discarding $1 to the script can loop over remaining file arguments.

LABORATORY RECORD

DATE..... EXP. No.....







DATE 22/09/26 EXP. No. 05

Title: Write a shell script and C program to perform the following string operations:

* To extract a substring from a given string.
* To find the length of a given string.

Compare the running Time of the above shell script and c program using time command.

Theory:

${#str}: It is a shell parameter expansion used to find the length of a string.

$pos: It is the starting poistion from where you want to extract after substring.

$len: Calculate length of the given string.

time ./string.sh: It is a linux command that measure how long the shell script takes to execute.

```
GNU nano 8.7.1
#include<stdio.h>
int main()
{
    char str[100];
    int start,length,i,count=0;

    printf("Enter string: ");
    fgets(str,100,stdin);

    while(str[count] != '\0' && str[count] != '\n'
    {
        count++;
    }

    printf("Length of string: %d\n ",count);
    printf("Enter starting positon: ");
    scanf("%d",&start);
    printf("Enter length: ");
    scanf("%d",&length);
    printf("Substring:  ");

    for(i=start; i<start+length && i< count; i++)
    {
        printf("%c",str[i]);
    }

    printf("\n");
    return 0;
}
```

```
vboxuser@harnoorkour:~$ my exp7.sh exp5.sh
vboxuser@harnoorkour:~$ bash exp5.sh
Enter a string: Hey feyre darling
Length of string : 17
Enter starting position: 2
Enter number of characters: 8
Substring: y feyre
vboxuser@harnoorkour:~$ time . exp5.sh
Enter a string: Hey feyre darling
Length of string : 17
Enter starting position: 2
Enter number of characters: 8
Substring: y feyre

real    0m8.222s
user    0m0.000s
sys     0m0.003s
vboxuser@harnoorkour:~$ gcc exp5.c -o exp5
vboxuser@harnoorkour:~$ ./exp5
Enter string: Hey feyre darling
Length of string: 17
 Enter starting positon: 2
Enter length: 8
Substring:  y feyre
vboxuser@harnoorkour:~$ gcc exp5.c -o exp5
vboxuser@harnoorkour:~$ time ./exp5
Enter string: Hey feyre darling
Length of string: 17
 Enter starting positon: 2
Enter length: 8
Substring:  y feyre

real    0m9.019s
user    0m0.002s
sys     0m0.007s
vboxuser@harnoorkour:~$
```

LABORATORY RECORD

DATE 29-09-26 EXP. No. 06

Title: Write a C program that takes, as a command line argument the number of megabytes of memory it will use and during execution it should consume that much memory. Observe memory usage during program execution using free command.

Theory:

gcc exp6.c -o exp6 : compiles the C program & creates the executable file.

./exp6 100 : Runs the program with a memory allocation request of 100MB

free -h : Displays system memory usage in human readable format.

watch -n 1 free : Displays updated memory usage every second.

```
vboxuser@harnoorkour:~$ gcc exp6.c -o exp6
vboxuser@harnoorkour:~$ ./exp6 100
100 MB memory allocated.
Program is  running... Press Ctrl+C to exit.
^C
vboxuser@harnoorkour:~$ watch -n 1 free -h
vboxuser@harnoorkour:~$
```

GNU nano 8.7.1 exp6.c

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>

main(    argc,      *argv[])

  mb;
if ( argc != 2)
{
  printf("Usage: %s <memory_in_MB>\n",argv[0]);
        1;
}

mb = atol(argv[1]);
    *memory =malloc(mb * 1024 * 1024);

if (memory == NULL)
{
  printf("Memory allocation failed\n");
        1;
}
```

GNU nano 8.7.1 exp

```
    *memory =malloc(mb * 1024 * 1024);

if (memory == NULL)
{
   printf("Memory allocation failed\n");
         1;
}

for (    i=0; i < mb * 1024 *1024 ; i+=4096)
{
   memory[i]= 1;
}
printf("%d MB memory allocated.\n", mb);
printf("Program is  running... Press Ctrl+C to exit.\n");

while (1)
{
   sleep(1);
}
      0;
```

```
Every 1.0s: free -h

               total        used        free      shared  buff/cache   available
Mem:           4.7Gi       1.2Gi       2.0Gi       51Mi       1.8Gi       3.5Gi
Swap:             0B          0B          0B
```

LABORATORY RECORD

DATE. 22/10/26

EXP. No. 07

Title: Write a CPU bound C-program and a I/O bound C program and observe the effect of their CPU share using top command and its variants.

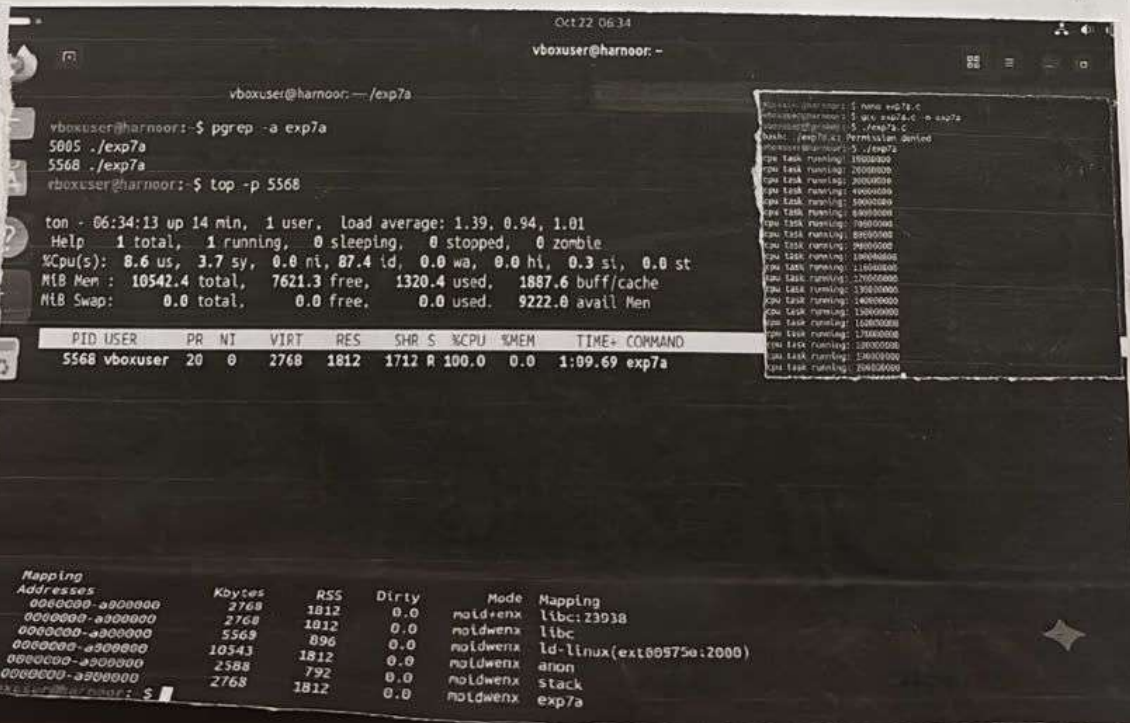
Theory:

CPU bound: A program is CPU bound when its execution speed is limited by the processing power of computer.

I/O bound: A program is I/O bound when its execution speed is limited by data transfer speed.

pgrep -a filename: Search for running processes whose command line matches filename and display their PID and full command line.

top -p PID: Displays the live status of a specific process using its process ID (PID).



DATE 22/10/24

EXP. No. 08

TITLE: Observe and understand the process map of the previous two lab programs using p-map command.

Theory:

process map: It is a system level document or data structure that details how a process virtual address space is organized and mapped to physical memory resources.

pmap <PID>: Displays the memory map of a process.

pmap -x <PID>: Displays the extended memory map of a process, including detail memory usage information.

```
 GNU nano 8.7.1                    file.c
#include<stdio.h>
#include<unistd.h>

int main(){
    FILE *fp;
    printf("Starting I/O task...\n");
    while(1) {
        fp = fopen("dummy.txt","W");
        if (fp != NULL) {
            fprintf(fp, "writting disk data\n");
            fclose(fp);
            printf("File operation done\n");
        }
        sleep(1);
    }
    return 0;
}
^G Help      ^O Write Out   ^F Where Is   ^K Cut       ^T Execute    ^C Location
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line
```

DATE 27/10/26 EXP. No. 09

Title: Write a program in C that creates a child process, waits for the termination of the child and lists its PID.

Theory:

fork(): creates a new process called the child process

PID == 0: indicates that the code is running the child process

getPID(): returns the process ID of the currently running process.

wait(NULL): Makes the parent process wait until the child process finishes execution.

pid: In parent process, it stores the PID of the child process

```
vb@harnoor:~$ gcc main.c -o exp9
vb@harnoor:~$ ./exp9
Parent Waiting for child to terminate...
Child running with PID: 12210
```

```
GNU nano 8.7.1
#include<stdio.h>
#include<unistd.h>

int main(){
  int var = 42;
  printf("PID: %d | Address: %p\n",
  getpid(), (void*)&var);
  fflush(stdout);

  while(1){
    sleep(1);
  }
  return 0;
}
```

LABORATORY RECORD

DATE 29/10/25

EXP. No. 10

Title: Observing virtual memory address across concurrent process in C.

Theory:

fork(): creates child process

&x: gains the memory of variables

%p: prints the memory address

Virtual memory: memory used by a process to store its data and instructions

Concurrent process: Parent and child can execute during overlapping periods.

```
vb@harnoor:~$ ./exp10 & ./exp10 & ./exp10 & ./exp10
[2] 15566
[3] 15567
[4] 15568
ID: 15567 | Address: 0x7ffe540035e4
ID: 15569 | Address: 0x7ffe2feae934
ID: 15568 | Address: 0x7ffe8eaad654
ID: 15566 | Address: 0x7fffea00cc44
```

```
GNU nano 8.7.1
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/wait.h>

int main(){
    p = fork();
    if(p==0){
     printf("Child running with p
     sleep(2);
     printf("Child Exiting...\n")
     exit(0);
    }

    else{
     printf("Parent Waiting for c
     terminated_pid = wait(
     printf("Parent Terminated Ch
    }
    return 0;
}
```